R.real.10

Complément Web

React Base

React

Pas un framework mais une bibliothèque :

https://fr.reactjs.org/

Une large communauté

Une approche composants, un DOM virtuel

La possibilité de développement avec des composants natifs sur périphériques mobiles (iOS, Android)

Rem : Aujourd’hui pas de natif

HTML

- Charger les bibliothèques

```
<script src="https://unpkg.com/react@18/umd/react.development.js"
crossorigin></script>
<script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"
crossorigin></script>
```

- Une partie de la DOM qui sera

```
<div id="app_container"></div>
```

- Charger le script `<script src="App.js"></script>`
- Associer un composant à la DOM gérée

```
const root = ReactDOM
    .createRoot(document.querySelector('#app_container'));
root.render(React.createElement(...));
```

Un composant

En React les composants sont des fonctions ou des classes.

Version fonction :

```
function Hello(props)  {

    return React
        .createElement('div', null, `Hello ${props.toWhat}`);
}
```

Un composant

En React les composants sont des fonctions ou des classes.

Version classe :

```
class Hello extends React.Component {
  render() {
    return React
        .createElement('div', null, `Hello ${this.props.toWhat}`);
  }
}
```

JSX JavaScript Syntax Extension

Permet une syntaxe proche de l'HTML nécessite un interpréteur

- Le composant

```
class Hello extends React.Component {
  render() {
    return <div> Hello {this.props.toWhat}</div>
  }
}
```

- L'interpréteur ici Babel en stand-alone

```
<script src="https://unpkg.com/babel-
standalone@6/babel.min.js"></script>
<script type="text/babel" src="App.js"></script>
```

Les composants se réutilisent et reçoivent de l'information

```
<div>
<Hello toWhat={'world'}/>
  <Hello toWhat={'jojo'}/>
</div>
```

Les composant disposent d’un état

L’état est géré sa modification va de manière asynchrone déclencher un render.

```
this.state = {….};
```

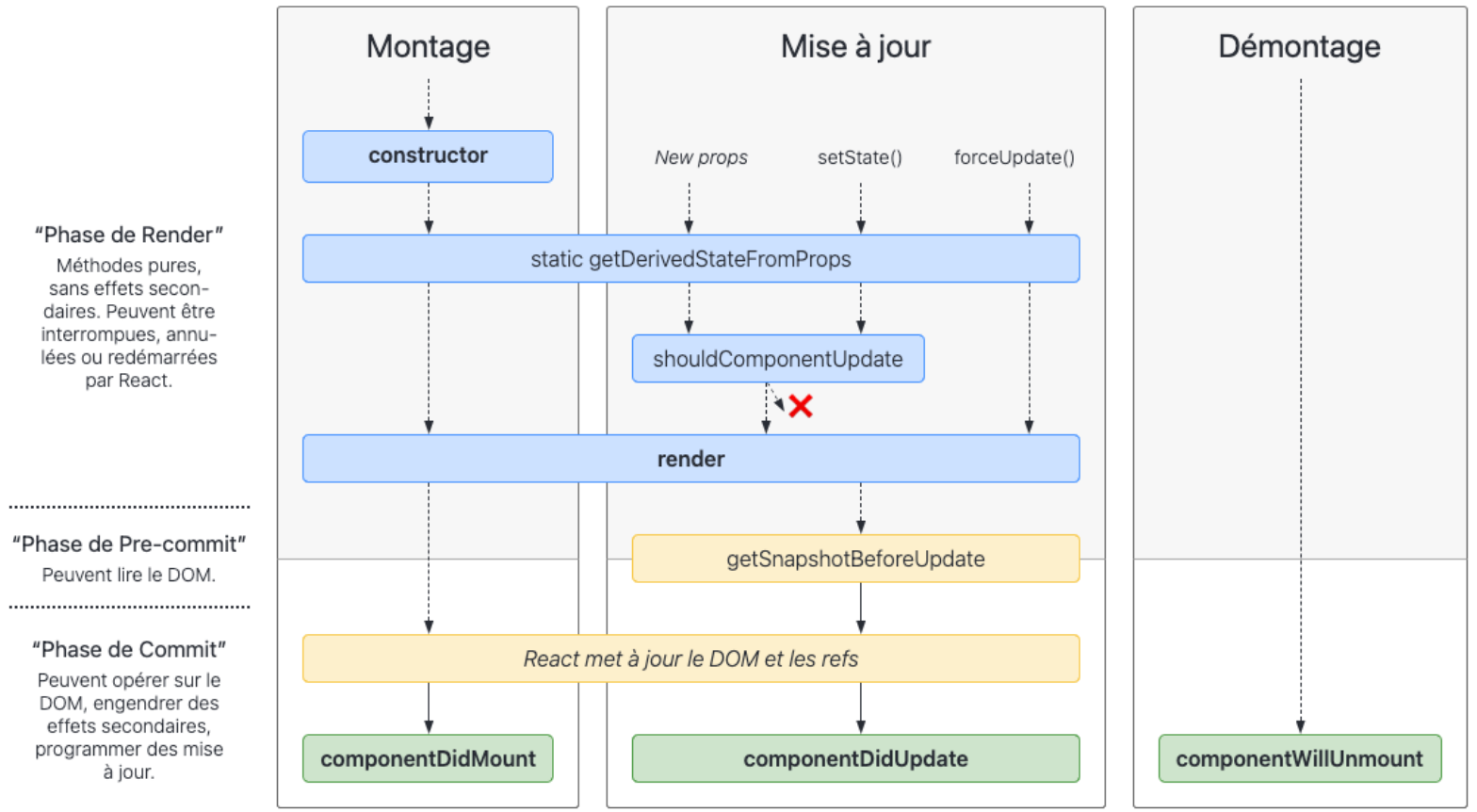
```
this.setState({ ….})
```

=> render

Attention à l’état

Attention, React va optimiser de manière asynchrone les changements d'états aussi la valeur du prochain état ne peut faire directement référence à la valeur précédante.

Cycle de vie


Montage
Mise à jour
Démontage
constructor
New props
setState()
forceUpdate()
static getDerivedStateFromProps
shouldComponentUpdate
render
getSnapshotBeforeUpdate
React met à jour le DOM et les refs
componentDidMount
componentDidUpdate
componentWillUnmount
"Phase de Render"
Méthodes pures, sans effets secondaires. Peuvent être interrompues, annulées ou redémarrées par React.
"Phase de Pre-commit"
Peuvent lire le DOM.
"Phase de Commit"
Peuvent opérer sur le DOM, engendrer des effets secondaires, programmer des mise à jour.

Les événements

Communication entre composants

Les props

<ChildComponent nom_prop={info} />

Dans le composant enfant this.props.nom_prop avec props reçu dans le contrôleur

Communication entre les composants (v1)

Passer une callback du parent vers l'enfant au travers d'une props

parentCallback = (info venant de l'enfant) => {traitement sur le parent}

<ChildComponent callback={this.parentCallback}> />

Rem : Nous avons une fonction dans une classe, à éviter

Communication entre les composants (v2)

Passer une **méthode** du parent vers l'enfant au travers d'une props

Dans le constructeur :
this.parentCallback = this.parentCallback.**bind**(this)

parentCallback (info venant de l'enfant)
{traitement sur le parent}

<ChildComponent callback={this.parentCallback}> />

Communication entre les composants (v3)

Passer une methode du parent vers l’enfant au travers d’une props

parentCallback (info venant de l’enfant)
{traitement sur le parent}

<ChildComponent callback={(info) => this.parentCallback(info)}> />

Communication entre composants

Le context : https://reactjs.org/docs/context.html

Offre la possibilité de communiquer au sein du même hiérarchie en partageant un état globale mais réduit la réutilisation des composants :